Supplementary materials: Deep learning-based design of broadband GHz complex and random metasurfaces

Tianning Zhang,[1] Chun Yun Kee,[1] Yee Sin Ang,[1, a)] and L. K. Ang[1, b)]

Science, Mathematics and Technology, Singapore University of Technology and Design

[a)]Electronic mail: yeesin_ang@sutd.edu.sg

[b)]Electronic mail: ricky_ang@sutd.edu.sg

I. MODEL ARCHITECTURE

A. Architecture of forward and inverse design model

The evaluation branch is adapted from the Resnet18 architecture which contains four layers and eight Resnet blocks. The usual pooling layer is removed. All the ReLU activation functions are replaced by LeakReLU activation functions. To avoid exploding/vanishing gradient problem, the last linear layer is followed by a *tanh* layer. The architecture of the forward design is illustrated in Fig.1.

The inverse design generation branch consists of five transposed convolution blocks with a *tanh* activation function. It converts the input 1D curve input (EM response) to a 2D image. The judging branch is made up of a series of regular convolution blocks, which is capable of comparing the input image and the design image by generator branch in terms of confidence of matching. The architecture of the forward design is illustrated in Fig.2 and Fig.3.

B. Other models in benchmarking

a. Random Forest Regression(RFR) The random forest regression is an ensemble learning method by constructing many decision trees at the training of datasets. RFR is a general purpose yet powerful model that require less resources to train and less parameters to be tuned in comparison to CNN based models. Its prediction is based on the average prediction of all trees.

b. ResNa The ResNa or ResnetNature is a neural network combining CNN with RNN, which is commonly used in natural language processing (NLP) and sequence modeling. Inspired by a paper[1], which a layer of gated recurrent units (GRU) is appended to the end of the prediction model, we construct our model with a small CNN network as the front layer and use a long short-term memory (LSTM) layer as the last layer, as illustrated in Fig.4. With the addition of LSTM layer, the model could provide a smoother and continuous output. This architecture is expected to perform better than normal CNN.

c. Resnet34S The Resnet34S is the deeper version of Resnet18S which is the base architecture of the evaluation branch in our MSDCNN. Having equipped with more parameters, its greater capacity is expected to be better than Resnet18S for larger datasets. However,

this also leads to more costly training in terms of computational resources and time.

d. RDN108000 This model refers to the Resnet18S model trained on a larger RDN dataset of 108k samples (4 times more than the default setting). It is able to provide highly accurate forward prediction for RDN datasets but less effective than PLG and PTN.

C. Higher order statistical analysis

Typically, a good agreement in the mean (1st) and variance (2nd) indicates an acceptable practical performance. The 4th order statistical measure like kurtosis can show more details about the matching between the predicted and real values. In the main text (see Fig.9 and 10), we have provided the mean-variance curve for model PTNRes34 and RDNResNa, which have better performance than the under-trained model RDN27000 in terms of mean and variance. Unfortunately, the discrepancy in kurtosis can not be removed completely as shown in Fig.**??**. Selected failed samples of using small RDN datasets (RDN27000) in training are shown in Fig.6 to illustrate that the prediction of the models (blue) can be very similar despite having very different EM responses from the real values (red), signaling the problem of mode collapse often encountered in GAN. This emphasizes again the difficulty to train on the complex and unconstrained RDN dataset.

II. DETAILS OF MSDCNN

To start, we train the evaluation branch as many as possible with the samples available as shown in Fig.7 The model we used is a modified Resnet18 with Leaky ReLU as the primary activation function. We scale the EM responses from [0,1] to [-1,1] and use the *tanh* layer as the output layer. The evaluation branch is trained on a training dataset of 27k samples and then is validated on a test dataset of 3k samples. The Mean Square Error (MSE) is chosen as the metric for evaluating the performance.

This generator branch is a part of a GAN (Fig.8) consists of a series of transpose-CNN layers. It starts from a $128 \times 1 \times 1$ curve, then gradually up-sampled to the final resolution of 16×16. The judging branch in GAN is the discriminating network, which compares the input image and predicted image by generator in terms of confidence of matching. Under the framework of GAN, the generator will learn to generate random binary images which

share the same statistics as the training dataset. With the PLG dataset, the generator will be capable of generating binary images which resemble polygon pattern.

To train this pure GAN model (without the evaluation branch), we need to feed a random vector of length 100 combined with actual image data. At the beginning, the generator model will generate a 16×16 matrix with values of entries between 0 and 1. After several epochs, it would gradually learn to generate a binary polygon image. To assess the similarity between the input image and the generated image, we use MSE and structural similarity index measure (SSIM)[2], which focuses on the brightness and contrast of the image. The pure GAN model will gradually learn the underlying patterns of polygon and capable of generating good quality PLG image when fed with a random sequence as shwon in Fig.9.

After training the pure GAN model and evaluation branch well, we merge them and train the whole model together. The evaluation branch is kept unchanged, the weights are frozen. The input now is a partially random vector of length 100 whose first 32 points are the training curve data and the last 68 points are still random. This combination is to overcome the problem of data inconsistency, which is first reported by Peurifoy et al.[3]. The training proceeds in this alternating manner that juggles between training of the generator and the judging branch. The generator is trained for one epoch by minimizing the curve loss and generator's loss. Subsequently, the judging branch is trained for one epoch by minimizing the discrimination loss and image loss. The remainder of the training continues by alternating between training of the generator and the judging branch. Note when one is being trained, the other is kept unchanged (the weights are frozen).

During the training, it is possible that the images produced by the generator contain voids which are not allowed in the PLG datasets. In this case, a processing step will be triggered to patch up the holes in the images before feeding to the judging branch. This further ensures that the patterns are consistent. In practice, regularization term can be fine-tuned via the hyper-parameters in the Adam optimizer.

A. Training Detail

Optimizer: Adam

GPU: NVIDIA Tesla P100

Forward training hyper parameter will list in table.II A

GAN Model Hyper parameter on PLG dataset "d_lr": 4e-05, "g_lr": 5e-04, "c_lr": 1e-04, "disc_iter": 3, "gen_iter": 5, "balance_coef": [0.5, 200, 0.05] The learning curve will show in figure.10

GAN Model Hyper parameter on RDN dataset "d_lr": 1e-04, "g_lr": 5e-04, "c_lr": 0.0001, "disc_iter": 2, "gen_iter": 5 The learning curve will show in figure.11

REFERENCES

[1]I. Sajedian, J. Kim, and J. Rho, "Finding the optical properties of plasmonic structures by image processing using a combination of convolutional neural networks and recurrent neural networks," Microsystems & nanoengineering **5**, 1–8 (2019).

[2]Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," IEEE transactions on image processing **13**, 600–612 (2004).

[3]J. Peurifoy, Y. Shen, L. Jing, Y. Yang, F. Cano-Renteria, B. G. DeLacy, J. D. Joannopoulos, M. Tegmark, and M. Soljačić, "Nanophotonic particle simulation and inverse design using artificial neural networks," Science advances **4**, eaar4206 (2018).

FIGURES

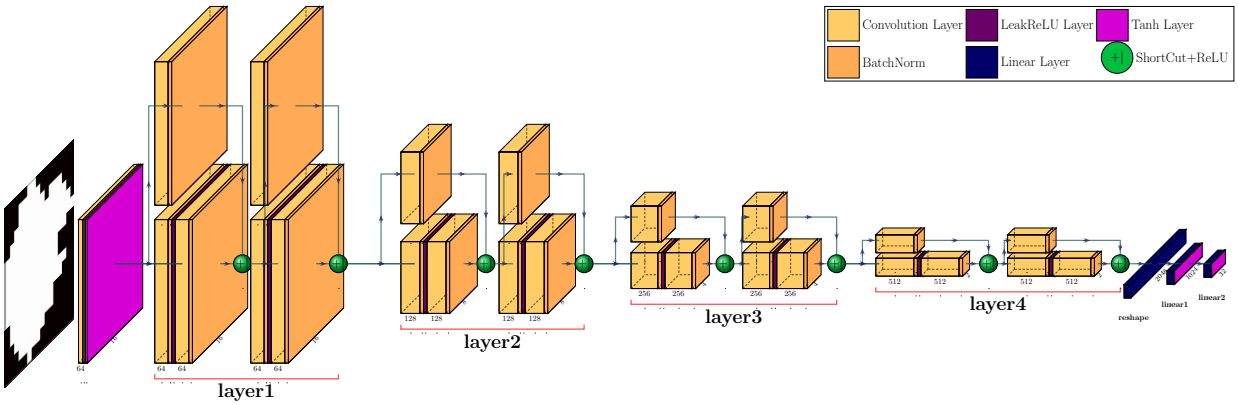


FIG. 1: Architecture of forward design prediction branch.

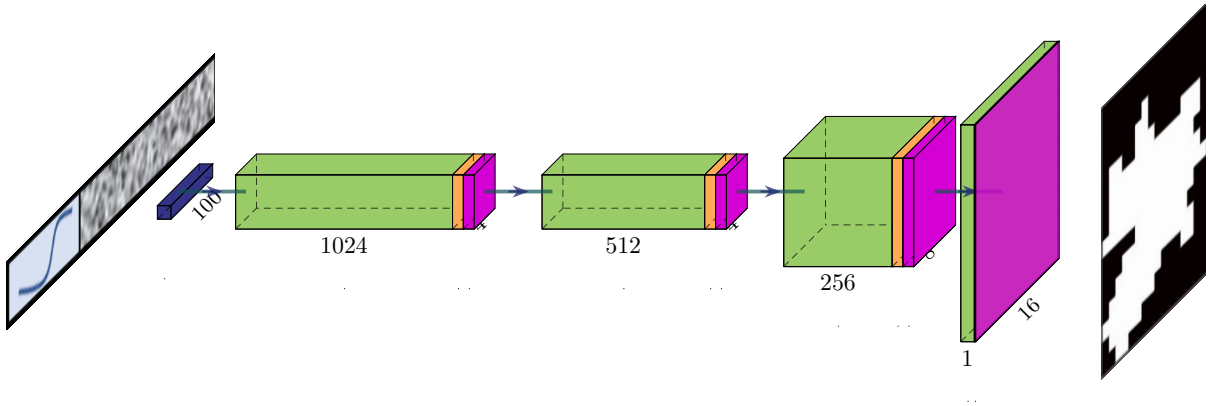


FIG. 2: Generator.

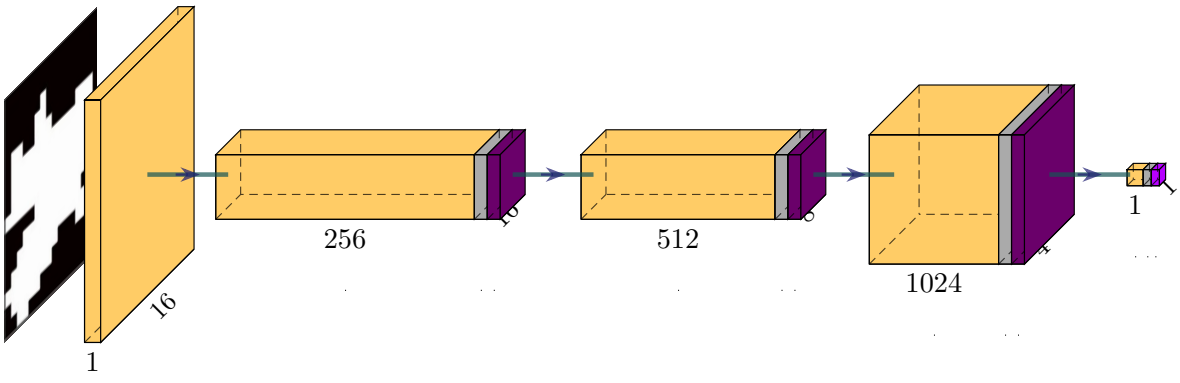


FIG. 3: Judger.

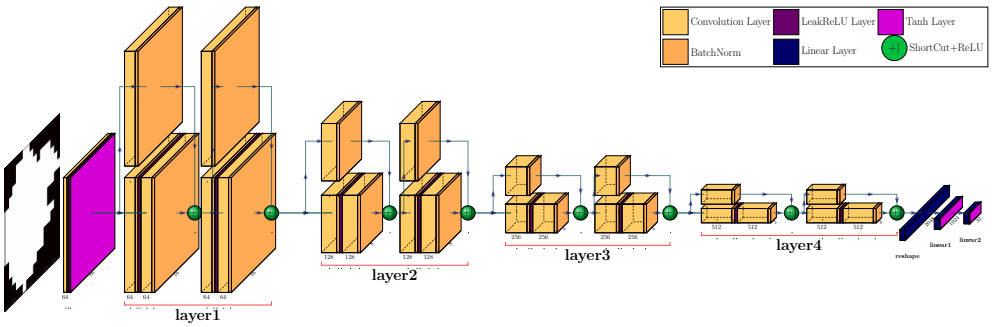


FIG. 4: Architecture of ResnetNature.

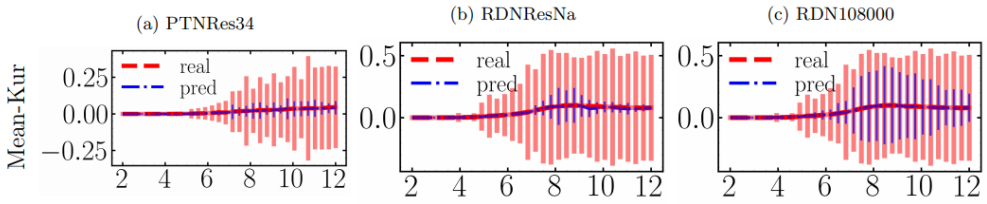


FIG. 5: Mean-Kurtosis analysis: (a) The Resnet34S model trained on PTN dataset has shown improved agreement in kurtosis. (b) The ResNa model trained on RDN dataset has better performance but discrepancy remains. (c) The MSDCNN model trained on RDN dataset with extended sample size 108k with good improvement in kurtosis.

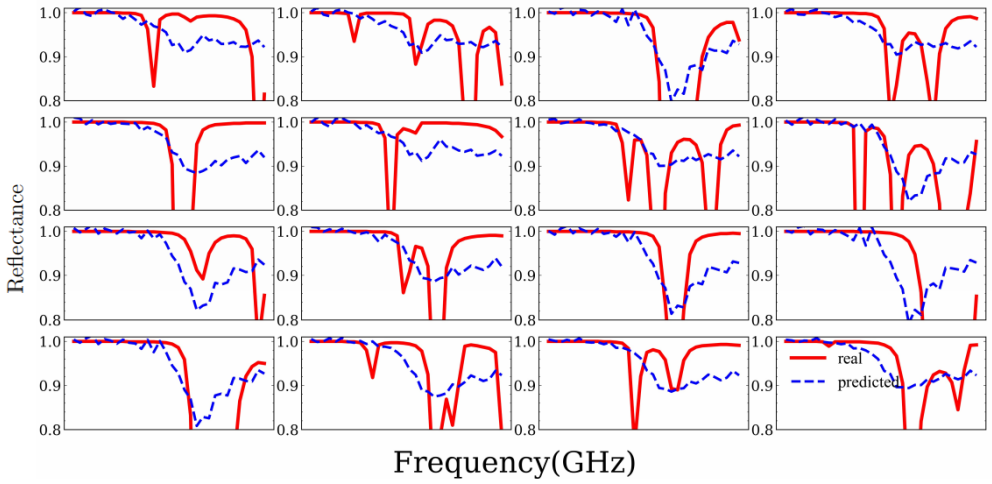


FIG. 6: Results for RDN27000 tested on RDN datasets. The model does not generalize well between the predicted (blue) and real (red) values. Some poor agreements are shown. This is similar to the mode collapse issue encountered in GAN.

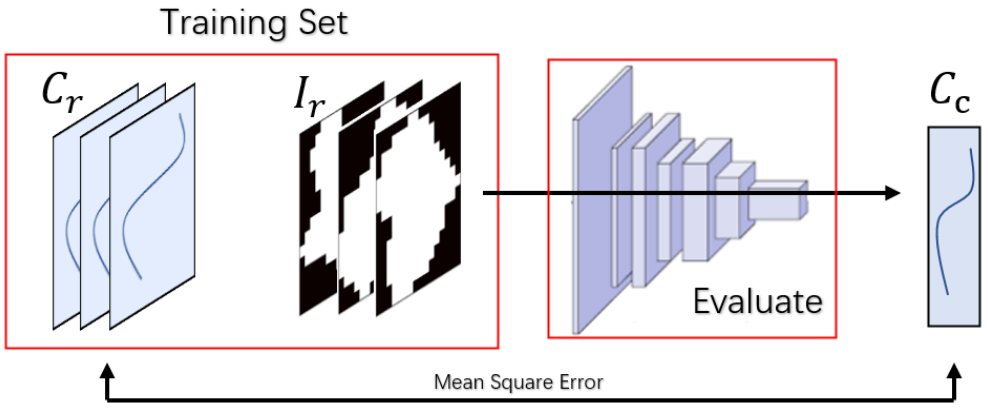


FIG. 7: Forward prediction design or evaluation branch

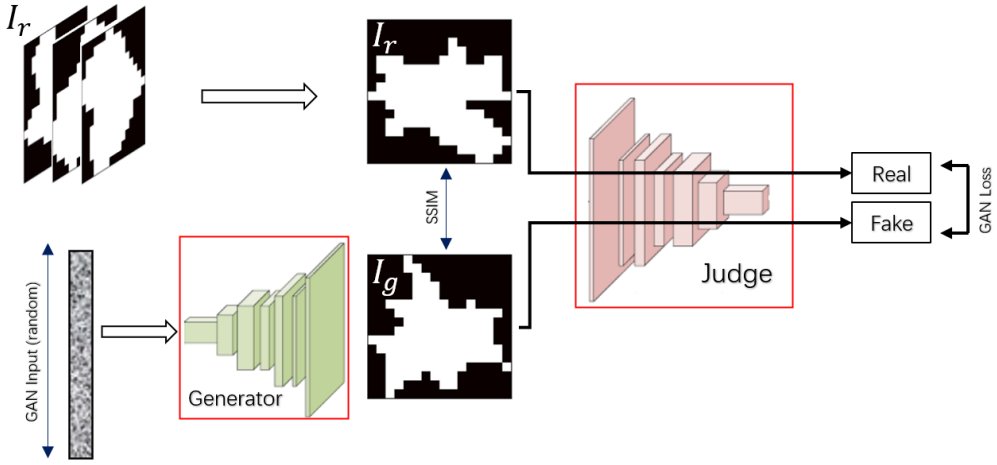


FIG. 8: GAN model (inverse design): the generator predicts an image from the input curve and the judging branch produces a quality score for this predicted image to measure the matching



FIG. 9: The performance of the inverse generator at 1000 and 10000 epochs



FIG. 10: Learning Curve for GAN at PLG dataset



FIG. 11: Learning Curve for RDN at PLG dataset

TABLES

TABLE I: Hyper parameter of model. Optimizer:Adam.GPU: NVIDIA Tesla P100

model	PLG27000	PTN27000	RDN27000	RDN80000	RDN85000	RDN90000
learning rate	2.20E-03	1.20E-03	1.20E-03	9.00E-06	2.00E-06	1.20E-04
batch size	6200	6200	6200	6200	6200	6200
model	RDN95000	RDN100000	RDN108000	PTNRes34	RDNResNa	
learning rate	2.00E-06	1.40E-05	5.00E-06	2.20E-03	3.20E-03	
batch size	6200	6200	6200	3800	1900	